



Birzeit University

Department of Electrical and Computer Engineering

Assignment 1: Telco Customer Churn Prediction

Using Machine Learning

Summer Semester, 2025/2026

Prepared by:

Donia Said 1222600

Supervised by:

Dr. Aziz Qaroush

Date : 3-9-2026

1. Introduction and Problem Definition

Customer churn prediction is a binary classification problem whose objective is to identify telecommunications customers who are likely to discontinue the service. This project develops a complete, leakage-safe machine-learning workflow covering exploratory data analysis, preprocessing, feature selection, baseline modeling, Logistic Regression, SVM, Random Forest, XGBoost, LightGBM, class-imbalance handling, cross-validation, hyperparameter tuning, and final evaluation.

The engineering goal is not simply to maximize Accuracy. Because failing to identify a customer who actually churns can have different business consequences from incorrectly flagging a retained customer, model selection will later consider Precision, Recall, F1-score, ROC-AUC, confusion matrices, and cross-validation stability. A fixed random seed (`random_state = 42`) is used throughout random operations to keep comparisons reproducible.

2. Dataset Description

The Telco Customer Churn dataset contains 7,043 customer records and 21 columns describing demographic, account, service, and billing characteristics. The target Churn contains Yes/No outcomes. `customerID` is a unique identifier, while the principal numerical variables are `tenure`, `MonthlyCharges`, and `TotalCharges`. The remaining predictors are mainly binary or multi-category service/account variables.

Item	Observed value	Modeling meaning
Dataset size	$7,043 \times 21$	Customer-level classification data
Target	Churn (Yes/No)	Binary outcome
Identifier	<code>customerID</code>	Unique; not a predictive characteristic
Numerical	<code>tenure</code> , <code>MonthlyCharges</code> , <code>TotalCharges</code>	Different scales; <code>TotalCharges</code> initially stored as text
Other predictors	Demographic, account, service, billing	Mostly categorical/binary

Table 1. Compact dataset overview.

Raw storage types were not treated as semantic feature types. `SeniorCitizen`, for example, is stored as `int64` but represents a binary indicator (0/1), whereas `TotalCharges` is conceptually numerical despite being loaded as a string. This distinction informs the preprocessing design in the next stage.

Task 1:

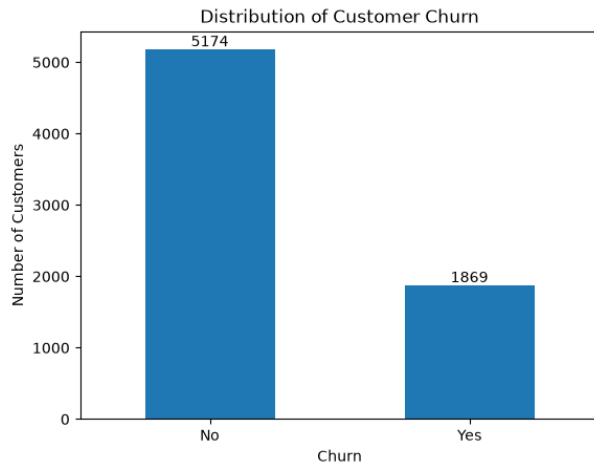
3. Exploratory Data Analysis

Initial inspection using `df.head()`, `df.shape`, and `df.dtypes` confirmed the dataset structure and data types before preprocessing.

3.1 Data Quality, Target Distribution, and Numerical Patterns

Standard null detection reported no NaN values, but a dedicated whitespace check found 11 invalid entries in `TotalCharges`. All 11 affected customers had `tenure = 0`. This pattern is consistent with newly registered customers whose accumulated charges had not yet been recorded, although the dataset does not explicitly state the cause. The whitespace values also explain why `TotalCharges` was read as a string. Therefore, numerical conversion and explicit treatment of these entries are required during preprocessing.

The target is moderately imbalanced: 5,174 customers (73.46%) did not churn and 1,869 (26.54%) churned. A naïve all-No classifier would therefore obtain 73.46% Accuracy while detecting no churners. This makes Accuracy insufficient on its own and motivates stratified splitting/cross-validation plus Precision, Recall, F1-score, and ROC-AUC in later evaluation.



Feature	Mean	Median	Range
tenure	32.37	29.00	0–72
MonthlyCharges	64.76	70.35	18.25–118.75
TotalCharges	2,283.30	1,397.48	18.80–8,684.80

Figure 1 and Table 2. Churn distribution and key numerical descriptive statistics.

Numerically, TotalCharges is strongly right-skewed: its mean (2,283.30) is substantially above its median (1,397.48). More importantly, churners had much shorter customer relationships: median tenure was 10 months versus 38 months for non-churners. Churners also had higher median MonthlyCharges (79.65 versus 64.43), but lower median TotalCharges (703.55 versus 1,683.60). The lower accumulated charges should be interpreted jointly with tenure because customers who leave earlier have fewer months in which charges can accumulate.

Measure	Churn = No	Churn = Yes
tenure — median	38.00	10.00
MonthlyCharges — median	64.43	79.65
TotalCharges — median	1,683.60	703.55

Table 3. Most decision-relevant numerical differences by churn status.

3.2 Categorical Churn Patterns

Several categorical variables showed substantial churn-rate differences. Contract produced the clearest gradient: 42.71% churn for month-to-month customers, 11.27% for one-year contracts, and only 2.83% for two-year contracts. Electronic check users had 45.29% churn, fiber-optic customers 41.89%, and customers without TechSupport 41.64%. In contrast, gender showed almost no difference (26.92% female versus 26.16% male), illustrating that not every available variable is equally informative.

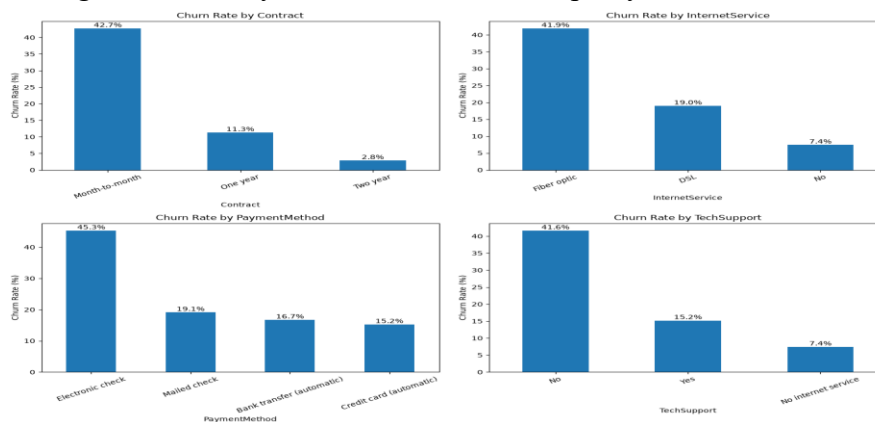
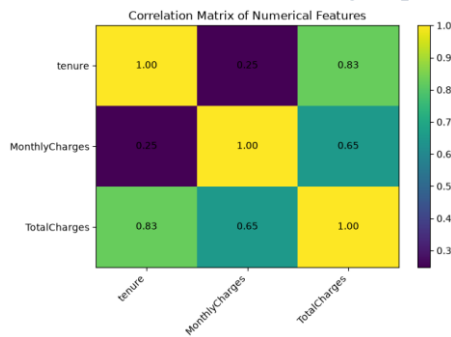


Figure 2. Churn rates across selected categorical characteristics.

These patterns are associations rather than causal effects. For example, the fiber-optic result may partly reflect other characteristics such as monthly charges or service combinations. The repeated 7.40% churn

rate for “No internet service” across multiple service variables also reflects structural dependence because these categories refer to the same underlying customer group.

3.3 Correlation, Outliers, and Modeling Implications



Correlation analysis supports the accumulated-charge interpretation. tenure and TotalCharges are strongly positively correlated ($r = 0.826$), MonthlyCharges and TotalCharges are moderately-to-strongly correlated ($r = 0.651$), while tenure and MonthlyCharges have a weaker relationship ($r = 0.248$). The strong tenure–TotalCharges relationship indicates possible redundancy that should be evaluated formally during feature selection rather than resolved by deleting a feature during EDA.

Figure 3. Pearson correlation matrix for numerical variables.

Using the global $1.5 \times \text{IQR}$ criterion, zero numerical outliers were detected for tenure, MonthlyCharges, or TotalCharges. Points beyond the whiskers in churn-specific boxplots arise because quartiles are calculated separately within each churn subgroup; they do not contradict the global IQR result and do not justify deleting observations.

3.4 EDA Conclusions and Decisions

- Exclude customerID from predictive modeling because it is a unique identifier.
- Convert TotalCharges to numeric and explicitly handle its 11 whitespace entries.
- Encode categorical variables and standardize numerical variables where required by scale-sensitive models.
- Use stratified splitting/cross-validation and multiple metrics because Churn is imbalanced (73.46% vs. 26.54%).
- Revisit tenure–TotalCharges redundancy and categorical relevance through formal training-only feature selection.
- Do not remove rows for numerical outliers because none were detected globally by the $1.5 \times \text{IQR}$ rule.

Importantly, the exploratory relationships above are used only to understand the data and motivate later experiments. Any feature-selection procedure used to build the final models will be recomputed using training data only, preserving the separation of the unseen test set.

Task 2:

4. Data Preprocessing

Preprocessing was designed to correct structural data issues while keeping all data-dependent transformations leakage-safe. The raw dataset was copied before modeling so that the original data remained unchanged. customerID was removed because it is unique for every observation and does not describe customer behavior. TotalCharges was then converted from text to a numerical variable using coercion; the 11 whitespace-only entries identified during EDA consequently became explicit missing values (NaN). After removing the identifier, the modeling table contained 7,043 rows and 20 columns, and TotalCharges had dtype float64 with exactly 11 missing values.

Dataset after ID removal	TotalCharges type	Missing TotalCharges	Predictors (X)	Target (y)
7,043 × 20	float64	11	7,043 × 19	7,043

Table 5. Verified preprocessing outputs before fitting any transformation.

The predictors were separated from Churn, producing 19 modeling features: three numerical variables (tenure, MonthlyCharges, TotalCharges) and 16 categorical/binary variables. The target was mapped

deterministically as No = 0 and Yes = 1, giving 5,174 observations in class 0 and 1,869 in class 1. This fixed mapping does not estimate any parameter from the data and therefore does not introduce leakage.

Input	Transformation	Engineering justification
customerID	Drop	Unique identifier; no generalizable customer signal.
TotalCharges	Convert to numeric	Whitespace values become explicit NaN; preserves valid charge values.
Numerical features	Median imputation	Only TotalCharges has missing values; median is robust to its right-skewed distribution.
Numerical features	StandardScaler	Places differently scaled variables on comparable scales; important for Logistic Regression and SVM.
Categorical features	OneHotEncoder	Nominal categories have no valid numeric order; handle_unknown="ignore" supports unseen categories.
Churn target	No→0, Yes→1	Defines churn as the positive class for later classification metrics.

Table 6. Preprocessing decisions and technical justification.

4.1 Leakage-Safe Preprocessing Design

A ColumnTransformer was prepared rather than fitting transformations directly on the complete dataset. The numerical branch applies SimpleImputer(strategy="median") followed by StandardScaler, while the categorical branch applies OneHotEncoder(handle_unknown="ignore"). The transformer was only defined at this stage; it was not fitted on the full dataset. In later modeling, these steps will be placed inside the model Pipeline so imputation, scaling, and encoding learn parameters from the training data (and from the training fold during cross-validation) rather than from the unseen test set.



Figure 4. Preprocessing workflow prepared for leakage-safe model pipelines.

At the end of Task 2, no rows had been removed for missing values or outliers, and no preprocessing statistics had been learned from the full dataset. This preserves all 7,043 customers for the subsequent stratified train/test split and ensures that the later feature-selection and model-comparison stages can be performed without contaminating the test set.

Task 3:

Before feature selection or any later data-dependent analysis, the 7,043 observations were divided into training and testing subsets using an 80/20 stratified split. The training set contains 5,634 customers with 19 predictors, while the unseen test set contains 1,409 customers with the same 19 predictors. The 80/20 ratio was selected to retain sufficient data for model development and cross-validation while preserving a meaningful hold-out sample for final evaluation.

Subset	X shape	y shape	Class 0: No Churn	Class 1: Churn
Full dataset	7,043 × 19	7,043	73.46%	26.54%
Training	5,634 × 19	5,634	73.46%	26.54%
Testing	1,409 × 19	1,409	73.46%	26.54%

Table 7. Verification of the 80/20 stratified train/test split.

Stratification was used because Churn is moderately imbalanced. The class proportions remained exactly 73.46% non-churn and 26.54% churn in both subsets, confirming that the split preserved the original target distribution. `random_state = 42` was fixed to make the partition reproducible.

Leakage control. From this point onward, `X_test` and `y_test` are treated as unseen hold-out data. Feature selection, preprocessing fitting, cross-validation, and hyperparameter tuning will use the training data only; the test set will be reserved for final model evaluation. This prevents information from the hold-out sample from influencing model-development decisions.

Task 4:

6. Feature Selection

Feature selection was evaluated on the training data only using two complementary approaches: Mutual Information (MI), a model-independent filter method, and L1-regularized Logistic Regression, an embedded method. After leakage-safe preprocessing, the 19 raw predictors expanded to 46 encoded features. MI highlighted contract type, tenure, technical-support/security indicators, fiber-optic service, monthly charges, electronic-check payment, and total charges; these patterns were consistent with the earlier EDA. L1 also emphasized contract duration, tenure, internet-service variables, and payment method, indicating partial agreement despite different ranking mechanisms.

Method	Accuracy	Precision	Recall	F1	ROC-AUC	Selected features
All features	0.8021	0.6530	0.5431	0.5923	0.8461	46
Mutual Information	0.8026	0.6539	0.5452	0.5939	0.8464	40 / 46
L1 selection	0.8028	0.6547	0.5445	0.5938	0.8460	31.8 / 46 avg.

Table 8. Leakage-safe 5-fold cross-validation comparison of feature-selection strategies.

Selection stability and interpretation. MI selected $k = 40$ in every outer fold, while L1 retained 33, 31, 31, 32, and 32 features (mean = 31.8), demonstrating a stable reduction of roughly 31% for L1. MI produced the highest Recall (0.5452), F1 (0.5939), and ROC-AUC (0.8464), whereas L1 produced the highest Accuracy (0.8028) and Precision (0.6547). However, the differences were extremely small: MI improved ROC-AUC over all features by only 0.0003, far below the observed cross-validation variability (ROC-AUC SD $\approx$ 0.0125–0.0126). Therefore, the methods should be interpreted as practically equivalent in predictive performance rather than as evidence of a meaningful gain.

Engineering decision. The full feature representation will be retained as the primary input for subsequent model comparison. This decision is data-driven: MI required 40 of 46 features for its best result, and the stronger L1 reduction did not improve ROC-AUC. Retaining all predictors avoids unnecessary information loss and allows each downstream algorithm to exploit the available variables according to its own learning mechanism. Feature selection was nevertheless validated formally using nested, stratified cross-validation, with preprocessing and selection fitted inside the training folds so the unseen test set remained untouched.

Task 5:

7. Baseline Model

A `DummyClassifier` using the most-frequent strategy was established as a deliberately simple baseline. Because 73.46% of the training observations belong to the non-churn class, this classifier predicts No Churn for every customer. Its purpose is not to provide a useful predictive system, but to quantify the performance that can be obtained from class prevalence alone and to provide a minimum reference for the real models.

Metric	Mean	Std
Accuracy	0.7346	0.0001
Precision	0.0000	0.0000
Recall	0.0000	0.0000
F1-score	0.0000	0.0000
ROC-AUC	0.5000	0.0000

Table 9. Stratified 5-fold cross-validation performance of the Dummy baseline.

Interpretation. The 0.7346 Accuracy is misleading when considered alone: the baseline detects no churners, producing zero Precision, Recall, and F1-score, while ROC-AUC = 0.5000 indicates no useful discrimination. Consequently, subsequent model selection must consider Recall, F1-score, and ROC-AUC in addition to Accuracy.

Task 6:

8. Initial Model Training and Comparison

Five classification algorithms were evaluated before hyperparameter tuning: Logistic Regression, Support Vector Machine (SVM), Random Forest, XGBoost, and LightGBM. The comparison used the full feature representation selected in Section 6, identical leakage-safe preprocessing, and the same stratified 5-fold cross-validation protocol. Keeping the evaluation procedure fixed isolates differences attributable to the learning algorithms rather than to different data splits or preprocessing choices.

Model	Accuracy	Precision	Recall	F1	ROC-AUC	AUC Std
Logistic Regression	0.8026	0.6547	0.5431	0.5930	0.8461	0.0125
LightGBM	0.7943	0.6428	0.5064	0.5663	0.8378	0.0087
XGBoost	0.7842	0.6137	0.5057	0.5544	0.8229	0.0084
Random Forest	0.7843	0.6237	0.4749	0.5387	0.8177	0.0114
SVM	0.8051	0.6820	0.4990	0.5761	0.7989	0.0181
Dummy Baseline	0.7346	0.0000	0.0000	0.0000	0.5000	0.0000

Table 10. Initial model comparison using stratified 5-fold cross-validation.

Comparative analysis. Logistic Regression produced the strongest overall balance, with the highest ROC-AUC (0.8461), Recall (0.5431), and F1-score (0.5930) among the five learning algorithms. SVM achieved the highest Accuracy (0.8051) and Precision (0.6820), but its lower Recall (0.4990) and ROC-AUC (0.7989) show why Accuracy alone is not an appropriate selection criterion for this moderately imbalanced churn problem. LightGBM ranked second by ROC-AUC (0.8378) and also showed the lowest variability among the strongest models (AUC SD = 0.0087), making it an important nonlinear candidate for later optimization. XGBoost and Random Forest exceeded the baseline substantially but did not match Logistic Regression.

Engineering interpretation and next decision. The results demonstrate that greater model complexity does not automatically improve generalization on this dataset. After one-hot encoding and scaling, a comparatively simple linear classifier captured the churn signal more effectively than the initial tree ensembles and SVM under the tested settings. Logistic Regression is therefore the current leading model, while LightGBM remains a strong nonlinear challenger. These results are pre-tuning estimates only; final model selection will be made after controlled hyperparameter optimization and class-imbalance experiments, with the unseen test set still reserved for final evaluation.

Task 7:

9. Hyperparameter Tuning

Hyperparameter optimization was performed for all five learning algorithms using the training data only. Logistic Regression used GridSearchCV, while SVM, Random Forest, XGBoost, and LightGBM were tuned with controlled search spaces. Stratified 5-fold cross-validation and ROC-AUC were used consistently as the optimization objective, and the hold-out test set remained untouched.

9.1 Logistic Regression Tuning

GridSearchCV evaluated regularization strength $C = \{0.01, 0.1, 1, 10, 100\}$ with the liblinear and lbfgs solvers. The best configuration used $C = 10$ and liblinear, producing a mean CV ROC-AUC of 0.8463. This is only +0.0002 above the untuned score (0.8461). The corresponding mean training ROC-AUC was 0.8499, close to validation performance, indicating stable generalization rather than overfitting. Scores also plateaued at $C = 10$ -100, showing that weaker regularization provided essentially no additional predictive benefit.

Setting	Untuned ROC-AUC	Tuned ROC-AUC	Change	Train ROC-AUC
Logistic Regression	0.8461	0.8463	+0.0002	0.8499

Table 11. Logistic Regression tuning result.

9.2 LightGBM Tuning

LightGBM has a larger hyperparameter space, so RandomizedSearchCV was used to explore 30 configurations efficiently rather than exhaustively evaluating every combination. The search varied the number of boosting iterations, learning rate, tree complexity, minimum child size, row subsampling, and feature subsampling. The best configuration combined 500 estimators with a low learning rate (0.01), seven leaves, maximum depth 3, minimum child samples 20, subsample 0.70, and colsample_bytree 1.00.

n_estimators	learning_rate	num_leaves	max_depth	min_child	subsample	colsample
500	0.01	7	3	20	0.70	1.00

Table 12. Best LightGBM hyperparameter configuration.

The tuned LightGBM achieved CV ROC-AUC = 0.8489, improving the untuned model by +0.0111 and slightly exceeding tuned Logistic Regression (0.8463). Its mean training ROC-AUC was 0.8674, giving a moderate train-validation gap of 0.0185. Importantly, more complex candidates were not automatically superior: one tested configuration reached training ROC-AUC = 0.9044 but validation ROC-AUC = 0.8473, a larger gap consistent with greater overfitting. The winning configuration therefore supports a conservative boosting pattern: many shallow trees learned gradually at a low learning rate.

Model	Before tuning	After tuning	Improvement
Logistic Regression	0.8461	0.8463	+0.0002
LightGBM	0.8378	0.8489	+0.0111

Table 13. Effect of hyperparameter tuning on the two selected candidates.

Engineering decision. Tuning improved four models meaningfully and left Logistic Regression nearly unchanged. XGBoost achieved the highest tuned CV ROC-AUC (0.8499), closely followed by LightGBM (0.8489), Logistic Regression (0.8463), Random Forest (0.8458), and SVM (0.8362). The results show that tuning can materially change model ranking, but gains must still be judged against cross-validation variability and business objectives.

9.3 SVM, Random Forest, and XGBoost Tuning

The remaining three algorithms were also tuned as required. Best SVM parameters were kernel = rbf, gamma = auto, and $C = 0.1$ (CV ROC-AUC = 0.8362). Random Forest selected n_estimators = 300, min_samples_split = 5, and max_depth = 5 (0.8458). XGBoost selected n_estimators = 300, max_depth = 2, and learning_rate = 0.03 (0.8499 in the search output; the standardized comparison run reported 0.8499/0.8499 approximately).

Model	Before Tuning ROC-AUC	After Tuning ROC-AUC	Improvement
-------	-----------------------	----------------------	-------------

XGBoost	0.8229	0.8499	+0.0270
LightGBM	0.8378	0.8489	+0.0111
Logistic Regression	0.8461	0.8463	+0.0002
Random Forest	0.8177	0.8458	+0.0281
SVM	0.7989	0.8362	+0.0373

Table 14. Hyperparameter tuning comparison for all five models.

Task 8:

10. Class Imbalance Handling

The training data contain a moderate class imbalance (73.46% non-churn versus 26.54% churn). To investigate whether the minority churn class could be detected more effectively, class weighting was applied to the two strongest tuned candidates: Logistic Regression and LightGBM. The `class_weight='balanced'` strategy increases the contribution of minority-class errors during learning without generating synthetic customers. All comparisons used the same leakage-safe preprocessing and stratified 5-fold cross-validation.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Tuned Logistic	0.8037	0.6557	0.5485	0.5968	0.8463
Balanced Logistic	0.7481	0.5164	0.8000	0.6275	0.8462
Tuned LightGBM	0.8051	0.6765	0.5104	0.5814	0.8489
Balanced LightGBM	0.7510	0.5199	0.8067	0.6322	0.8483

Table 15. Effect of class weighting on the tuned candidate models.

Effect of balancing. Class weighting substantially increased sensitivity to churn. Logistic Regression Recall rose from 0.5485 to 0.8000, while LightGBM Recall increased from 0.5104 to 0.8067. F1-score also improved from 0.5968 to 0.6275 for Logistic Regression and from 0.5814 to 0.6322 for LightGBM. These gains were accompanied by the expected reductions in Accuracy and Precision because the balanced models classified more customers as likely churners and therefore produced more false-positive retention alerts.

ROC-AUC interpretation. ROC-AUC remained almost unchanged after weighting: 0.8463 to 0.8462 for Logistic Regression and 0.8489 to 0.8483 for LightGBM. Thus, weighting did not materially improve the models' underlying ranking/discrimination ability; instead, it shifted the classification behavior toward the minority class. This distinction is important because the benefit of balancing is primarily operational - detecting more true churners - rather than producing a higher global discrimination score.

Engineering decision. Balanced LightGBM provides the strongest churn-oriented operating point, achieving the highest Recall (0.8067) and F1-score (0.6322) while retaining ROC-AUC = 0.8483. It will therefore be carried forward as a primary final candidate alongside the unweighted tuned LightGBM, which preserves higher Accuracy and Precision and the best cross-validation ROC-AUC (0.8489). Keeping both candidates allows the final unseen-test evaluation to distinguish between overall discrimination and the business trade-off between missed churners and unnecessary retention actions.

10.1 SMOTE and Confusion-Matrix Comparison

A second imbalance-handling approach, SMOTE, was evaluated inside an imblearn Pipeline so synthetic minority samples were created only within the training folds. This avoids leakage and provides a direct comparison with the tuned unweighted and class-weighted LightGBM models.

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
Tuned LightGBM	0.8051	0.6765	0.5104	0.5814	0.8489
Balanced LightGBM	0.7510	0.5199	0.8067	0.6322	0.8483
SMOTE LightGBM	0.7749	0.5581	0.7344	0.6340	0.8456

Table 16. Comparison of imbalance-handling strategies.

Class weighting produced the highest Recall (0.8067), while SMOTE produced the highest F1-score (0.6340) and a middle-ground operating point. On the hold-out test set, the confusion matrices were: Tuned LightGBM TN=941, FP=94, FN=179, TP=195; Balanced LightGBM TN=752, FP=283, FN=72, TP=302; SMOTE LightGBM TN=812, FP=223, FN=95, TP=279. Thus, both imbalance methods reduce missed churners, with class weighting being the most aggressive and SMOTE offering a more moderate FP/FN trade-off.

Task 9:

11. Cross-Validation and Metric Selection

The best tuned pipeline for each of the five algorithms was re-evaluated under the same stratified 5-fold cross-validation protocol. ROC-AUC is used as the primary ranking metric because it evaluates discrimination across thresholds and is less misleading than Accuracy under class imbalance; Recall and F1 remain important secondary metrics for churn detection.

Model	Accuracy Mean	Accuracy Std	Precision Mean	Precision Std	Recall Mean	Recall Std	F1 Mean	F1 Std	ROC-AUC Mean	ROC-AUC Std
XGBoost	0.8048	0.0100	0.6718	0.0270	0.5177	0.0213	0.5846	0.0206	0.8499	0.0117
LightGBM	0.8051	0.0079	0.6765	0.0237	0.5104	0.0242	0.5814	0.0188	0.8489	0.0110
Logistic Regression	0.8037	0.0125	0.6557	0.0275	0.5485	0.0387	0.5968	0.0297	0.8463	0.0128
Random Forest	0.7980	0.0080	0.6972	0.0333	0.4247	0.0184	0.5273	0.0165	0.8458	0.0103
SVM	0.7991	0.0079	0.6730	0.0282	0.4749	0.0225	0.5563	0.0172	0.8362	0.0132

Table 17. Five-model cross-validation comparison after tuning.

Task 10:

12. Top-3 ROC Curves, Confusion Matrices, and Business Cost

The top three models by tuned CV ROC-AUC were XGBoost, LightGBM, and Logistic Regression. Their untouched-test ROC-AUC values were 0.8450, 0.8473, and 0.8411 respectively. The ROC curves show very similar ranking ability, with LightGBM slightly leading on the hold-out set.

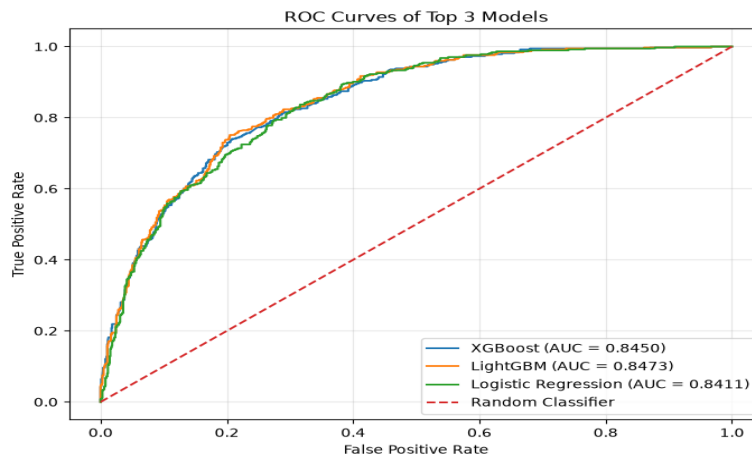


Figure 5. ROC curves for the top three tuned models on the test set.

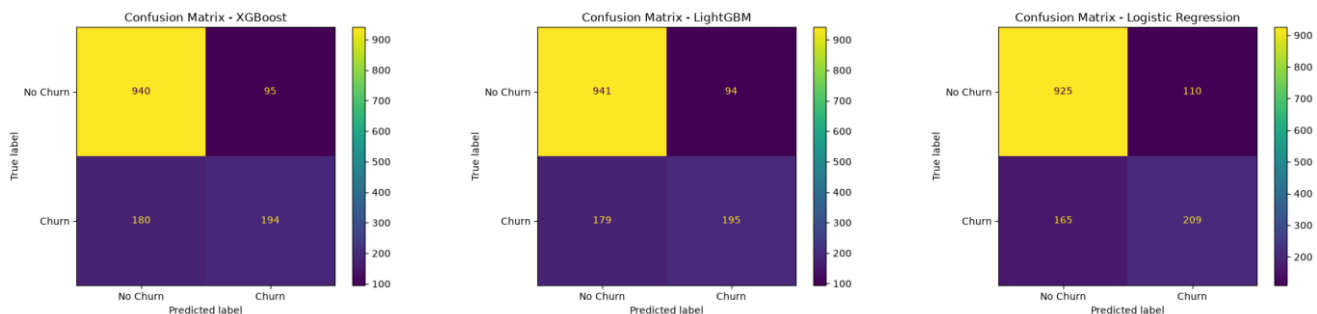


Figure 6. Confusion matrices for XGBoost, LightGBM, and Logistic Regression.

Business-cost scenario. A false negative costs \$500 because a true churner is missed, while a false positive costs \$50 because a retention action is unnecessarily offered. Therefore, FN is ten times more costly than FP, making Recall

especially important and motivating a lower decision threshold than the default 0.50. Thresholds were selected using out-of-fold training probabilities only, then applied once to the test set.

Model	Threshold	FP	FN	TP	TN	Precision	Recall	F1-score	Total Cost (\$)
Logistic Regression	0.08	566	12	362	469	0.3901	0.9679	0.5561	34,300
LightGBM	0.07	618	9	365	417	0.3713	0.9759	0.5380	35,400
XGBoost	0.10	537	19	355	498	0.3980	0.9492	0.5608	36,350

Table 18. Test-set performance at training-selected cost-sensitive thresholds.

Under this cost structure, Logistic Regression yields the lowest test cost (\$34,300), despite not having the highest Accuracy or ROC-AUC. This demonstrates that the highest-Accuracy model is not necessarily the best business model: the preferred operating point depends on the relative consequences of FP and FN errors.

Task 11:

13. Data Leakage Challenge

An intentional leakage experiment was performed to show why preprocessing and feature selection must be fitted inside cross-validation. In the incorrect setup, preprocessing and Mutual Information feature selection were fitted on all training observations before the folds were created. This allows information from each validation fold to influence the selected representation. In the correct setup, both preprocessing and feature selection were placed inside the Pipeline and re-fitted independently within each training fold.

Approach	Mean ROC-AUC	Std ROC-AUC
Incorrect - Feature Selection Before CV	0.8411	0.0138
Correct - Feature Selection Inside Pipeline	0.8426	0.0139

Table 19. Intentional data-leakage experiment.

The leaked score was not inflated in this particular run; it was slightly lower. This is still leakage because leakage is defined by information flow, not by whether the reported metric increases. Any result from the incorrect procedure is untrustworthy because the validation folds influenced model development. The remedy is to place every learned transformation—imputation, scaling, encoding, feature selection, and resampling—inside the cross-validation Pipeline.

Task 12:

Model Comparison

A single best pipeline was selected for each required algorithm using cross-validated ROC-AUC as the common model-selection criterion. All five final pipelines retained the full feature representation and used tuned hyperparameters. Class weighting and SMOTE were evaluated separately, but neither improved ROC-AUC enough to replace the tuned unbalanced pipeline as the common ranking choice. This keeps the final comparison consistent while the recall gains from imbalance handling are discussed separately.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
XGBoost	0.8048	0.6718	0.5177	0.5846	0.8499
LightGBM	0.8051	0.6765	0.5104	0.5814	0.8489
Logistic Regression	0.8037	0.6557	0.5485	0.5968	0.8463
Random Forest	0.7980	0.6972	0.4247	0.5273	0.8458
SVM	0.7991	0.6730	0.4749	0.5563	0.8362

Table 20. Final Model Performance Comparison - 5-fold cross-validation (best pipeline per model).

Model	Feature Set	Tuning	Imbalance	Selection Rationale
-------	-------------	--------	-----------	---------------------

Logistic Regression	All features	Tuned	None	Highest ROC-AUC version
SVM	All features	Tuned	None	Highest ROC-AUC version
Random Forest	All features	Tuned	None	Highest ROC-AUC version
XGBoost	All features	Tuned	None	Highest ROC-AUC version
LightGBM	All features	Tuned	None	Highest ROC-AUC version

Table 21. Configuration used for each final pipeline.

Model	Default ROC-AUC	Tuned ROC-AUC	Improvement
XGBoost	0.8229	0.8499	+0.0270
LightGBM	0.8378	0.8489	+0.0111
Logistic Regression	0.8461	0.8463	+0.0002
Random Forest	0.8177	0.8458	+0.0281
SVM	0.7989	0.8362	+0.0373

Table 22. Default versus tuned cross-validation ROC-AUC.

Tuning improved every model. The largest gain was observed for SVM (+0.0373), followed by Random Forest (+0.0281) and XGBoost (+0.0270). Logistic Regression changed only marginally (+0.0002), indicating that its default configuration was already close to the best setting for this dataset.

Feature selection produced little practical benefit: all features achieved ROC-AUC 0.8461, Mutual Information 0.8464, and L1 selection 0.8460. Therefore, the full feature set was retained. For imbalance handling, tuned LightGBM achieved ROC-AUC 0.8489, class-weighted LightGBM 0.8483, and SMOTE LightGBM 0.8456. Balancing substantially increased recall (about 0.51 to 0.81 with class weighting and 0.73 with SMOTE), but reduced precision and accuracy; thus it changes the operating trade-off more than ranking quality.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
LightGBM	0.8062	0.6747	0.5214	0.5882	0.8473
XGBoost	0.8048	0.6713	0.5187	0.5852	0.8450
Logistic Regression	0.8048	0.6552	0.5588	0.6032	0.8411
Random Forest	0.7921	0.6753	0.4171	0.5157	0.8398
SVM	0.7921	0.6484	0.4733	0.5471	0.8299

Table 23. Final performance on the untouched test set.

Model	CV ROC-AUC	Test ROC-AUC	Test - CV
XGBoost	0.8499	0.8450	-0.0048
LightGBM	0.8489	0.8473	-0.0016
Logistic Regression	0.8463	0.8411	-0.0052
Random Forest	0.8458	0.8398	-0.0060
SVM	0.8362	0.8299	-0.0062

Table 24. Cross-validation versus final test ROC-AUC.

The consistently small CV-to-test gaps suggest generally stable generalization, while a small degree of optimistic selection bias may remain because hyperparameter selection and subsequent CV evaluation reused the same training data and fold structure. Importantly, the final test set remained completely untouched until final evaluation.

Task 13:

Which Model Should We Deploy?

Recommended deployment model: cost-sensitive Logistic Regression with decision threshold = 0.08. The deployment decision is based on the complete experimental evidence rather than accuracy alone. Under the stated business costs, a false negative costs \$500 and a false positive costs \$50, so missing a true churner is ten times more expensive than contacting a customer who would not churn.

Model	Threshold	FP	FN	Precision	Recall	F1	Total Cost
Logistic Regression	0.08	566	12	0.3901	0.9679	0.5561	\$34,300
LightGBM	0.07	618	9	0.3713	0.9759	0.5380	\$35,400
XGBoost	0.10	537	19	0.3980	0.9492	0.5608	\$36,350

Table 25. Cost-sensitive test comparison using thresholds selected from training cross-validation.

Logistic Regression is preferred because it produced the lowest measured business cost (\$34,300), only 12 false negatives, and recall of 0.9679 at the training-selected threshold. It also offers strong interpretability, low computational complexity, fast training and prediction, and straightforward operational monitoring. Its CV ROC-AUC (0.8463) and test ROC-AUC (0.8411) differ by only 0.0052, supporting acceptable generalization.

Compared with LightGBM, Logistic Regression sacrifices only three additional false negatives (12 versus 9) but generates 52 fewer false positives (566 versus 618), reducing total cost from \$35,400 to \$34,300. LightGBM has the highest standard-threshold test ROC-AUC (0.8473) and excellent robustness, but it is more complex and less transparent. Compared with XGBoost, Logistic Regression produces fewer false negatives (12 versus 19) and a lower total cost (\$34,300 versus \$36,350), while also being substantially easier to interpret and cheaper to train and serve. Random Forest and SVM are less attractive alternatives because their final test ROC-AUC values are lower (0.8398 and 0.8299, respectively), and neither offers a compensating business-cost advantage in the evaluated results.

Class-imbalance experiments confirmed that recall can be increased through class weighting or SMOTE, but this comes with more false positives and lower precision/accuracy. Rather than selecting a balanced model solely for recall, the recommended operating policy directly incorporates the asymmetric error costs through threshold selection. Reliability is strengthened by the stratified train/test split, stratified 5-fold cross-validation, leakage-safe pipelines, an explicit leakage challenge, and threshold selection using out-of-fold training predictions rather than the final test set.

Required Final Analysis

- 1. Best overall discrimination:** XGBoost led CV ROC-AUC (0.8499), while LightGBM led test ROC-AUC (0.8473).
- 2. Best recall:** Balanced LightGBM reached about 0.81 recall at the default threshold; cost-sensitive LightGBM reached 0.9759.
- 3. Best ROC-AUC:** XGBoost was best in CV; LightGBM was best on the untouched test set.
- 4. Imbalance handling:** Class weighting and SMOTE increased recall substantially but reduced precision/accuracy; ROC-AUC changed little.
- 5. Tuning:** Tuning benefited SVM, Random Forest, XGBoost, and LightGBM most; Logistic Regression was already near optimum.
- 6. Feature selection:** MI and L1 did not materially improve ROC-AUC, so all features were retained.
- 7. Leakage:** Fitting learned transformations before CV invalidates validation independence even when the leaked score is not higher.
- 8. Primary metric:** ROC-AUC is appropriate for ranking models; recall/F1 and business cost are essential for operational churn decisions.
- 9. Business cost:** Because FN costs 10x FP, lower thresholds are justified; Logistic Regression at 0.08 achieved the lowest measured cost.
- 10. Deployment:** Deploy cost-sensitive Logistic Regression because it offers the best measured cost/recall trade-off with high interpretability and low complexity.

Conclusion

The project establishes a complete leakage-safe churn-prediction workflow from exploratory analysis through final deployment selection. Boosted trees provide the strongest ranking performance, while Logistic Regression remains highly competitive and becomes the preferred deployment choice when the specified asymmetric business costs are incorporated. The final recommendation therefore reflects predictive performance, error costs, interpretability, computational efficiency, robustness, class imbalance, and evaluation reliability rather than a single headline metric.